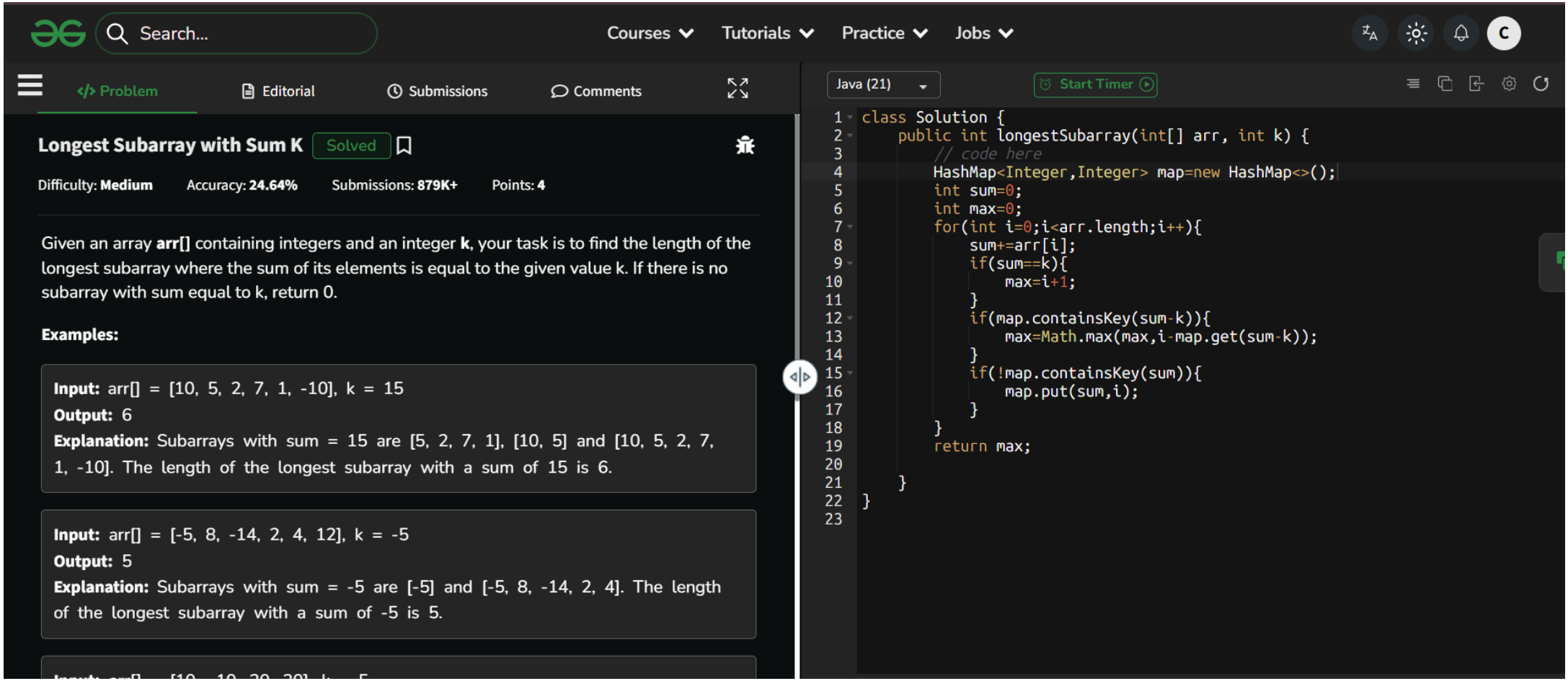
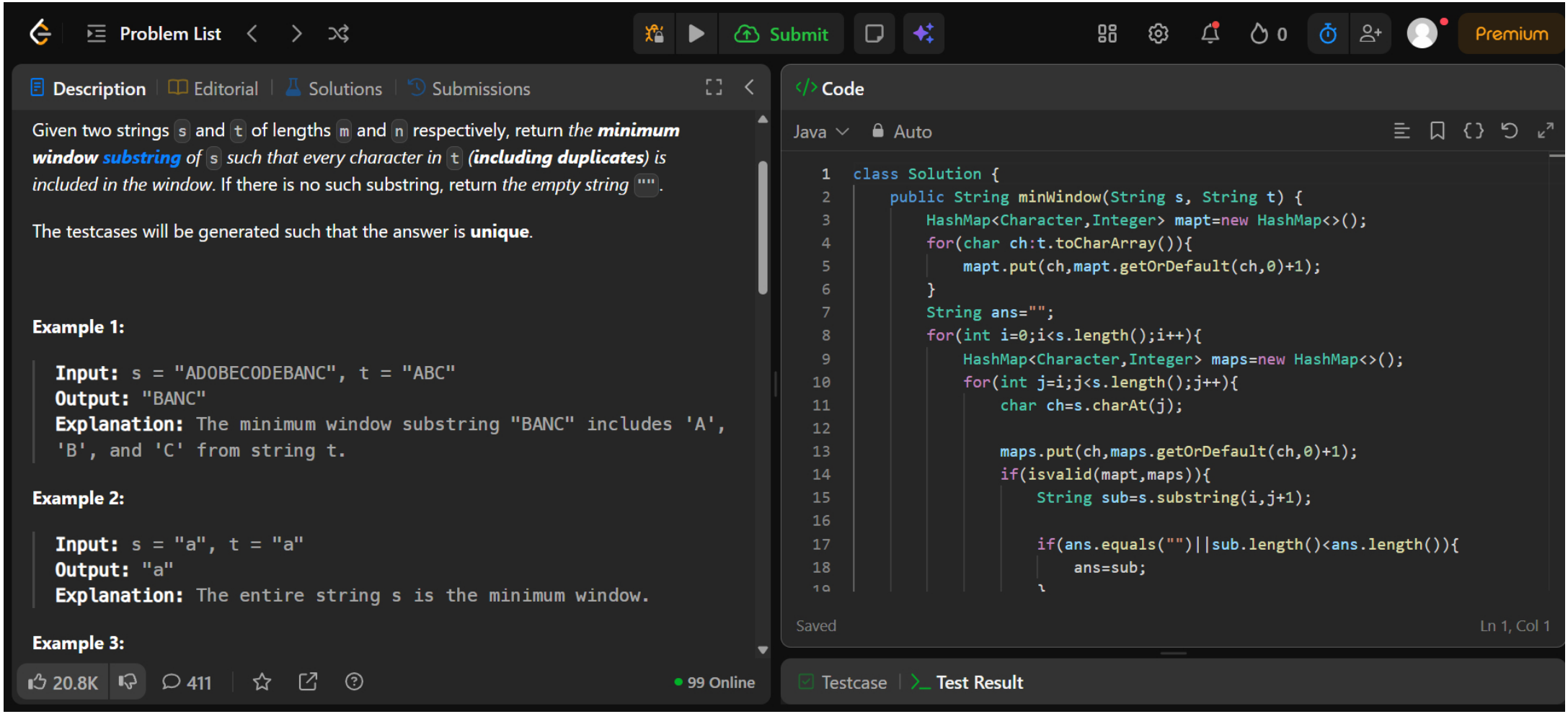

Search...
Courses
Tutorials
Practice
Jobs
C
Problem
Editorial
Submissions
Comments
Longest Subarray with Sum K
Solved
Difficulty: Medium
Accuracy: 24.64%
Submissions: 879K+
Points: 4
Given an array arr[] containing integers and an integer k, your task is to find the length of the longest subarray where the sum of its elements is equal to the given value k. If there is no subarray with sum equal to k, return 0.
Examples:
Input: arr[] = [10, 5, 2, 7, 1, -10], k = 15
Output: 6
Explanation: Subarrays with sum = 15 are [5, 2, 7, 1], [10, 5] and [10, 5, 2, 7, 1, -10]. The length of the longest subarray with a sum of 15 is 6.
Input: arr[] = [-5, 8, -14, 2, 4, 12], k = -5
Output: 5
Explanation: Subarrays with sum = -5 are [-5] and [-5, 8, -14, 2, 4]. The length of the longest subarray with a sum of -5 is 5.
Java (21)
Start Timer
class Solution {
public int longestSubarray(int[] arr, int k) {
// code here
HashMap<Integer,Integer> map=new HashMap<>();
int sum=0;
int max=0;
for(int i=0;i<arr.length;i++){
sum+=arr[i];
if(sum==k){
max=i+1;
}
if(map.containsKey(sum-k)){
max=Math.max(max,i-map.get(sum-k));
}
if(!map.containsKey(sum)){
map.put(sum,i);
}
}
return max;
}
}


Problem List
Submit
0
Premium
Description
Editorial
Solutions
Submissions
Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "".
The testcases will be generated such that the answer is unique.
Example 1:
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
Example 2:
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
Example 3:
20.8K
411
99 Online
Code
Java
Auto
class Solution {
public String minWindow(String s, String t) {
HashMap<Character,Integer> mapt=new HashMap<>();
for(char ch:t.toCharArray()){
mapt.put(ch,mapt.getOrDefault(ch,0)+1);
}
String ans="";
for(int i=0;i<s.length();i++){
HashMap<Character,Integer> maps=new HashMap<>();
for(int j=i;j<s.length();j++){
char ch=s.charAt(j);
maps.put(ch,maps.getOrDefault(ch,0)+1);
if(isvalid(mapt,maps)){
String sub=s.substring(i,j+1);
if(ans.equals("")||sub.length()<ans.length()){
ans=sub;
Saved
Ln 1, Col 1
Testcase
Test Result

Description | Editorial | Solutions | Submissions

56. Merge Intervals

Solved

Medium | Topics | Companies

Given an array of `intervals` where `intervals[i] = [start_i, end_i]`, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input.*

Example 1:

> **Input:** intervals = [[1,3],[2,6],[8,10],[15,18]]
> **Output:** [[1,6],[8,10],[15,18]]
> **Explanation:** Since intervals [1,3] and [2,6] overlap, merge them into [1,6].

Example 2:

> **Input:** intervals = [[1,4],[4,5]]
> **Output:** [[1,5]]

Code

Java | Auto

```java
class Solution {
    public int[][] merge(int[][] intervals) {
        ArrayList<int[]> al=new ArrayList<>();
        Arrays.sort(intervals,(a,b)->a[0]-b[0]);
        int currS=intervals[0][0];
        int currE=intervals[0][1];
        for(int i=1;i<intervals.length;i++){
            int nextS=intervals[i][0];
            int nextE=intervals[i][1];
            if(nextS<=currE){
                currE=Math.max(currE,nextE);
            }
            else{
                al.add(new int[]{currS,currE});
                currS=nextS;
                currE=nextE;
            }
        }
        al.add(new int[]{currS,currE});
```

Saving...

Ln 1, Col 1

Description | Editorial | Solutions | Submissions

703. Kth Largest Element in a Stream

Solved

Easy | Topics | Companies

You are part of a university admissions office and need to keep track of the `kth` highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer `k`, maintains a stream of test scores and continuously returns the `k`th highest test score **after** a new score has been submitted. More specifically, we are looking for the `k`th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer `k` and the stream of test scores `nums`.
- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the kth largest element in the pool of test scores so far.

6.6K | 262 | 18 Online

Code

Java | Auto

```java
class KthLargest {
    PriorityQueue<Integer> q;
    int k;

    public KthLargest(int k, int[] nums) {
        q=new PriorityQueue<>();
        this.k=k;
        for(int n:nums){
            q.add(n);

            if(q.size()>k){
                q.poll();
            }
        }
    }
    public int add(int val) {
        q.add(val);
        while(q.size()>k){
            q.poll();
```

Saving... Ln 15, Col 6

Testcase | Test Result

Description | Editorial | Solutions | Submissions

142. Linked List Cycle II

Solved

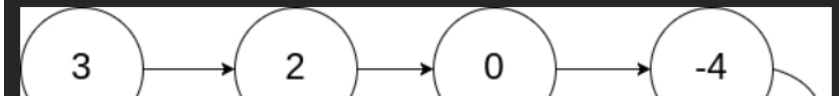
Medium | Topics | Companies

Given the `head` of a linked list, return *the node where the cycle begins. If there is no cycle, return* `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (**0-indexed**). It is `-1` if there is no cycle. **Note that** `pos` **is not passed as a parameter**.

Do not modify the linked list.

Example 1:



Code

Java | Auto

```java
/**
 * Definition for singly-linked list.
 * class ListNode {
 *     int val;
 *     ListNode next;
 *     ListNode(int x) {
 *         val = x;
 *         next = null;
 *     }
 * }
 */
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow=head;
        ListNode fast=head;
        while(fast!=null&&fast.next!=null){
            slow=slow.next;
            fast=fast.next.next;
```

Saved

Ln 1, Col 1

Description | Editorial | Solutions | Submissions

207. Course Schedule

Solved

Medium | Topics | Companies | Hint

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [$a_i$, $b_i$]` indicates that you **must** take course b_i first if you want to take course a_i.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is
```

Code

Java | Auto

```java
class Solution {
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        List<List<Integer>> al=new ArrayList<List<Integer>>();
        for(int i=0;i<numCourses;i++){
            al.add(new ArrayList<Integer>());
        }
        int[] indegree=new int[numCourses];
        for(int edge[]:prerequisites){
            int u=edge[1];
            int v=edge[0];
            al.get(u).add(v);
            indegree[v]++;
        }
        Queue<Integer> q=new LinkedList<>();
        for(int i=0;i<numCourses;i++){
            if(indegree[i]==0){
                q.add(i);
```

Saved

Ln 1, Col 1

Testcase | Test Result

Description | Editorial | Solutions | Submissions

322. Coin Change

Solved

Medium | Topics | Companies

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the **fewest** number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

21.4K | 289

Code

Java | Auto

```java
class Solution {
    public int coinChange(int[] coins, int amount) {
        int n=coins.length;
        int[][] dp=new int[n+1][amount+1];
        for(int i=0;i<=n;i++){
            dp[i][0]=0;
        }
        for(int j=0;j<=amount;j++){
            dp[0][j]=amount+1;
        }
        for(int i=1;i<=n;i++){
            for(int j=1;j<=amount;j++){
                int nottake=dp[i-1][j];
                int take=amount+1;
                if(coins[i-1]<=j){
                    take=1+dp[i][j-coins[i-1]];
                }
                dp[i][j]=Math.min(take,nottake);
            }
```

Restored from local | Upgrade to Cloud Saving | Ln 1, Col 1

Testcase | Test Result